

Practical Session 12

Task 1

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

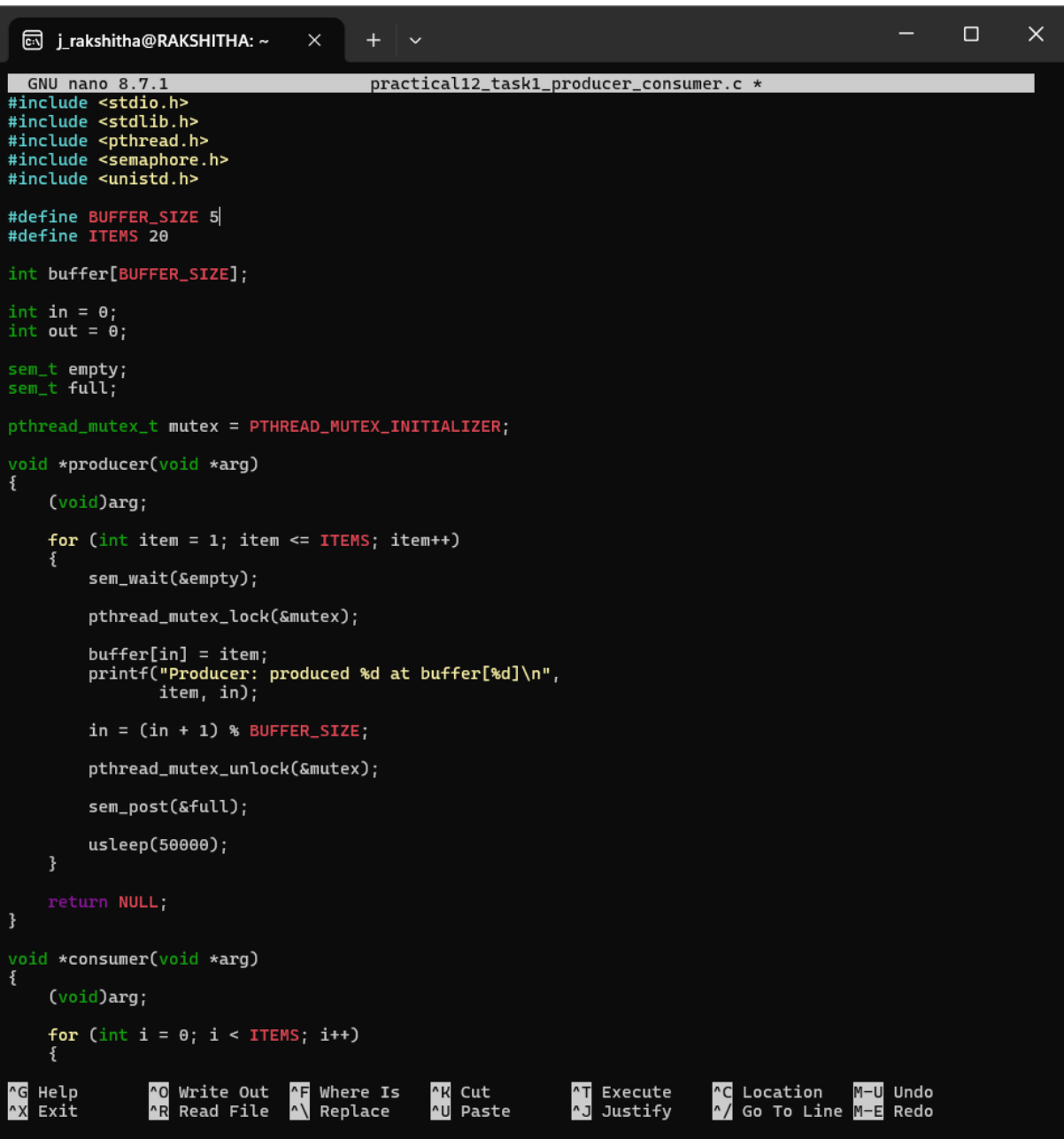
```
j_rakshitha@RAKSHITHA: ~$ nano practical12_task1_producer_consumer.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -pthread practical12_task1_producer_consumer.c -o practical12_task1_producer_consumer
j_rakshitha@RAKSHITHA:~$ ./practical12_task1_producer_consumer
=====
Practical Session 12 - Task 1
Producer-Consumer using Semaphores
=====

Buffer size: 5
Items: 20

Producer: produced 1 at buffer[0]
Consumer: consumed 1 from buffer[0]
Producer: produced 2 at buffer[1]
Consumer: consumed 2 from buffer[1]
Producer: produced 3 at buffer[2]
Producer: produced 4 at buffer[3]
Consumer: consumed 3 from buffer[2]
Producer: produced 5 at buffer[4]
Consumer: consumed 4 from buffer[3]
Producer: produced 6 at buffer[0]
Producer: produced 7 at buffer[1]
Consumer: consumed 5 from buffer[4]
Producer: produced 8 at buffer[2]
Consumer: consumed 6 from buffer[0]
Producer: produced 9 at buffer[3]
Producer: produced 10 at buffer[4]
Consumer: consumed 7 from buffer[1]
Producer: produced 11 at buffer[0]
Producer: produced 12 at buffer[1]
Consumer: consumed 8 from buffer[2]
Producer: produced 13 at buffer[2]
Consumer: consumed 9 from buffer[3]
Producer: produced 14 at buffer[3]
Consumer: consumed 10 from buffer[4]
Producer: produced 15 at buffer[4]
Consumer: consumed 11 from buffer[0]
Producer: produced 16 at buffer[0]
Consumer: consumed 12 from buffer[1]
Producer: produced 17 at buffer[1]
Consumer: consumed 13 from buffer[2]
Producer: produced 18 at buffer[2]
Consumer: consumed 14 from buffer[3]
Producer: produced 19 at buffer[3]
Consumer: consumed 15 from buffer[4]
Producer: produced 20 at buffer[4]
Consumer: consumed 16 from buffer[0]
Consumer: consumed 17 from buffer[1]
```

```
Consumer: consumed 18 from buffer[2]
Consumer: consumed 19 from buffer[3]
Consumer: consumed 20 from buffer[4]

Producer-consumer execution completed successfully.
j_rakshitha@RAKSHITHA:~$ |
```



```
GNU nano 8.7.1 practical12_task1_producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg)
{
    (void)arg;

    for (int item = 1; item <= ITEMS; item++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer: produced %d at buffer[%d]\n",
            item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        usleep(50000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITEMS; i++)
    {

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical12_task1_producer_consumer.c
    sem_wait(&full);

    pthread_mutex_lock(&mutex);

    int item = buffer[out];

    printf("Consumer: consumed %d from buffer[%d]\n",
           item, out);

    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);

    sem_post(&empty);

    usleep(80000);
}

return NULL;
}

int main(void)
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    printf("=====\n");
    printf("Practical Session 12 - Task 1\n");
    printf("Producer-Consumer using Semaphores\n");
    printf("=====\n");

    printf("\nBuffer size: %d\n", BUFFER_SIZE);
    printf("Items: %d\n\n", ITEMS);

    if (sem_init(&empty, 0, BUFFER_SIZE) != 0)
    {
        perror("sem_init empty");
        return 1;
    }

    if (sem_init(&full, 0, 0) != 0)
    {
        perror("sem_init full");
        sem_destroy(&empty);
        return 1;
    }

    if (pthread_create(&producer_thread, NULL,
                      producer, NULL) != 0)
    {
        perror("pthread_create producer");
    }

    |
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
        perror("pthread_create producer");
        return 1;
    }

    if (pthread_create(&consumer_thread, NULL,
                     consumer, NULL) != 0)
    {
        perror("pthread_create consumer");
        return 1;
    }

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("\nProducer-consumer execution completed successfully.\n");

    return 0;
}
```

```
j_rakshitha@RAKSHITHA: ~ x + v
j_rakshitha@RAKSHITHA:~$ nano practical12_task1_producer_consumer.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -pthread practical12_task1_producer_consumer.c -o practical12_task1_producer_consumer
j_rakshitha@RAKSHITHA:~$ ./practical12_task1_producer_consumer
=====
Practical Session 12 - Task 1
Producer-Consumer using Semaphores
=====

Buffer size: 10
Items: 20

Producer: produced 1 at buffer[0]
Consumer: consumed 1 from buffer[0]
Producer: produced 2 at buffer[1]
Consumer: consumed 2 from buffer[1]
Producer: produced 3 at buffer[2]
Producer: produced 4 at buffer[3]
Consumer: consumed 3 from buffer[2]
Producer: produced 5 at buffer[4]
Consumer: consumed 4 from buffer[3]
Producer: produced 6 at buffer[5]
Producer: produced 7 at buffer[6]
Consumer: consumed 5 from buffer[4]
Producer: produced 8 at buffer[7]
Consumer: consumed 6 from buffer[5]
Producer: produced 9 at buffer[8]
Producer: produced 10 at buffer[9]
Consumer: consumed 7 from buffer[6]
Producer: produced 11 at buffer[0]
Producer: produced 12 at buffer[1]
Consumer: consumed 8 from buffer[7]
Producer: produced 13 at buffer[2]
Consumer: consumed 9 from buffer[8]
Producer: produced 14 at buffer[3]
Producer: produced 15 at buffer[4]
Consumer: consumed 10 from buffer[9]
Producer: produced 16 at buffer[5]
Consumer: consumed 11 from buffer[0]
Producer: produced 17 at buffer[6]
Producer: produced 18 at buffer[7]
Consumer: consumed 12 from buffer[1]
Producer: produced 19 at buffer[8]
Producer: produced 20 at buffer[9]
Consumer: consumed 13 from buffer[2]
Consumer: consumed 14 from buffer[3]
Consumer: consumed 15 from buffer[4]
Consumer: consumed 16 from buffer[5]
Consumer: consumed 17 from buffer[6]
Consumer: consumed 18 from buffer[7]
Consumer: consumed 19 from buffer[8]
Consumer: consumed 20 from buffer[9]

Producer-consumer execution completed successfully.
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical12_task1_producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 10
#define ITEMS 20

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *producer(void *arg)
{
    (void)arg;

    for (int item = 1; item <= ITEMS; item++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Producer: produced %d at buffer[%d]\n",
            item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        usleep(50000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITEMS; i++)
    {
```

j_rakshitha@RAKSHITHA: ~

practical12_task1_producer_consumer.c

```
sem_wait(&full);

pthread_mutex_lock(&mutex);

int item = buffer[out];

printf("Consumer: consumed %d from buffer[%d]\n",
      item, out);

out = (out + 1) % BUFFER_SIZE;

pthread_mutex_unlock(&mutex);

sem_post(&empty);

usleep(80000);
}

return NULL;
}

int main(void)
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    printf("=====\n");
    printf("Practical Session 12 - Task 1\n");
    printf("Producer-Consumer using Semaphores\n");
    printf("=====\n");

    printf("\nBuffer size: %d\n", BUFFER_SIZE);
    printf("Items: %d\n\n", ITEMS);

    if (sem_init(&empty, 0, BUFFER_SIZE) != 0)
    {
        perror("sem_init empty");
        return 1;
    }

    if (sem_init(&full, 0, 0) != 0)
    {
        perror("sem_init full");
        sem_destroy(&empty);
        return 1;
    }

    if (pthread_create(&producer_thread, NULL,
                     producer, NULL) != 0)
    {
        perror("pthread_create producer");
    }
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^I Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```

        perror("pthread_create producer");
        return 1;
    }

    if (pthread_create(&consumer_thread, NULL,
                     consumer, NULL) != 0)
    {
        perror("pthread_create consumer");
        return 1;
    }

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("\nProducer-consumer execution completed successfully.\n");

    return 0;
}

```

Task 2

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```

j_rakshitha@RAKSHITHA:~$ nano practical12_task2_deadlock.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -pthread practical12_task2_deadlock.c -o practical12_task2_
deadlock
j_rakshitha@RAKSHITHA:~$ ./practical12_task2_deadlock
=====
Practical Session 12 - Task 2
Deadlock Demonstration
=====
Thread 1: locking Resource 1...
Thread 1: Resource 1 locked.
Thread 2: locking Resource 2...
Thread 2: Resource 2 locked.
Thread 1: waiting for Resource 2...
Thread 2: waiting for Resource 1...
^C
j_rakshitha@RAKSHITHA:~$

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical12_task2_deadlock.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1_function(void *arg)
{
    (void)arg;

    printf("Thread 1: locking Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 locked.\n");
    sleep(1);

    printf("Thread 1: waiting for Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_function(void *arg)
{
    (void)arg;

    printf("Thread 2: locking Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 locked.\n");
    sleep(1);

    printf("Thread 2: waiting for Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked.\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main(void)
[ Read 73 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^C Location  M-U Undo
              ^_ Go To Line M-E Redo
```

```

}

pthread_t thread1;
pthread_t thread2;

printf("=====\n");
printf("Practical Session 12 - Task 2\n");
printf("Deadlock Demonstration\n");
printf("=====\n\n");

pthread_create(&thread1, NULL, thread1_function, NULL);
pthread_create(&thread2, NULL, thread2_function, NULL);

pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

pthread_mutex_destroy(&resource1);
pthread_mutex_destroy(&resource2);

printf("\nDeadlock demonstration completed.\n");

return 0;
}
|

```

```

j_rakshitha@RAKSHITHA:~$ nano practical12_task2_ordered.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -pthread practical12_task2_ordered.c -o practical12_task2_o
rdered
j_rakshitha@RAKSHITHA:~$ ./practical12_task2_ordered
=====
Practical Session 12 - Task 2
Deadlock Prevention - Resource Ordering
=====

Thread 2: locking Resource 1...
Thread 2: Resource 1 locked.
Thread 1: locking Resource 1...
Thread 2: locking Resource 2...
Thread 2: Resource 2 locked.
Thread 2: using both resources.
Thread 2: resources released.
Thread 1: Resource 1 locked.
Thread 1: locking Resource 2...
Thread 1: Resource 2 locked.
Thread 1: using both resources.
Thread 1: resources released.

Deadlock prevented successfully.
j_rakshitha@RAKSHITHA:~$

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical12_task2_ordered.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1_function(void *arg)
{
    (void)arg;

    printf("Thread 1: locking Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Resource 1 locked.\n");

    sleep(1);

    printf("Thread 1: locking Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Resource 2 locked.\n");
    printf("Thread 1: using both resources.\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: resources released.\n");

    return NULL;
}

void *thread2_function(void *arg)
{
    (void)arg;

    printf("Thread 2: locking Resource 1...\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Resource 1 locked.\n");

    sleep(1);

    printf("Thread 2: locking Resource 2...\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Resource 2 locked.\n");
    printf("Thread 2: using both resources.\n");

    pthread_mutex_unlock(&resource2);

    [ Read 81 lines ]
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```

pthread_mutex_unlock(&resource1);

printf("Thread 2: resources released.\n");

return NULL;
}

int main(void)
{
    pthread_t thread1;
    pthread_t thread2;

    printf("=====\n");
    printf("Practical Session 12 - Task 2\n");
    printf("Deadlock Prevention - Resource Ordering\n");
    printf("=====\n\n");

    pthread_create(&thread1, NULL, thread1_function, NULL);
    pthread_create(&thread2, NULL, thread2_function, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\nDeadlock prevented successfully.\n");

    return 0;
}
|

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

[^]C
[^]/